

CogPath: A Constraint-Guided Context-Reduction Framework for LLM-Based Test Generation

Anonymous Author(s)

Abstract

Large language models (LLMs) have shown promise in automated unit test generation, yet achieving high branch coverage for complex methods remains challenging. We attribute this limitation not to model capability alone, but to a fundamental cognitive misalignment between what prompts expose and what branch reachability requires: implicit cross-method preconditions are hidden, while branch-irrelevant code is over-provided. To address this misalignment, we propose COGPATH, a constraint-guided context-reduction framework that externalizes hidden path conditions as Constraint-Hints and isolates branch-relevant dependencies via coverage-driven Backward Slicing, within an iterative generate–execute–repair loop. Our empirical evaluation, conducted on 2971 methods with cyclomatic complexity greater than 10 from 14 open-source projects, demonstrates that COGPATH achieves 12.8% higher line coverage and 10.2% higher branch coverage compared to the state-of-the-art, with improvements reaching 32.3% and 20% respectively on the most complex project.

Keywords

Unit Test Generation, Large Language Models, Software Testing, Program Slicing

1 INTRODUCTION

Automated unit test generation plays a critical role in improving software reliability and reducing developer effort. Traditional techniques, such as symbolic execution and search-based software testing (SBST), have made substantial progress in systematically exploring input spaces and maximizing structural coverage [2, 5, 11, 12]. However, these approaches often face fundamental limitations in scalability and in handling complex program semantics: They struggle to capture implicit preconditions that arise from intricate object states and inter-procedural interactions, where the conditions required to reach a target path are established outside the local method scope through sequences of prior method calls [?].

Recent advances demonstrate that large language models (LLMs) can directly synthesize executable unit tests, significantly alleviating manual effort [???]. To handle complex methods and improve coverage, subsequent research mainly pursues two directions. The first is *feedback-driven refinement*: CoverUp [3] iteratively targets untested regions using dynamic coverage analysis, while YATE [?] repairs failing tests to enhance validity. The second direction emphasizes *structural decomposition and path sensitivity*. To simplify complex logic, HITS [18] prompts LLMs to decompose methods into fragments. Advancing this using program analysis, SymPrompt [18] extracts branch constraints from Control Flow Graphs (CFGs), whereas Panta [7] fuses static CFG analysis with dynamic coverage feedback to identify uncovered paths. Building upon this

trajectory, JUnitGenie [9] achieves strict path sensitivity by isolating the minimal, path-specific context required for targeted generation.

Despite these promising results, a substantial coverage gap remains, rooted not in a lack of feedback or prompting sophistication, but in a deeper mismatch between the information exposed to the LLM and what is actually required to cover complex target branches. ① One important dimension of this mismatch is *What to Satisfy*. In many cases, branch reachability depends on implicit preconditions scattered across long dependency chains, requiring the model to infer hidden constraints through multiple variable assignments and state transitions before it can synthesize a valid test. Existing approaches typically provide only the target branch structure and compress constraint inference and test synthesis into a single generation step, without any explicit intermediate output or independent verification of the inferred constraints. As a result, the LLM can easily skip or compress critical reasoning steps, producing tests that appear plausible but fail to satisfy the actual conditions required to reach the target branch. This problem becomes particularly severe when the underlying dependency chain is long or semantically complex. ② The second dimension concerns *What to Attend To*. Existing methods usually expose the entire method body to the LLM, even though only a small subset of statements has real data or control dependence on the target branch. The resulting context is therefore structurally misaligned with the branch-coverage objective: coverage-critical dependencies are diluted by irrelevant code, attention is spread across non-essential statements, and the model must reason under a representation that over-emphasizes what is visible rather than what is truly necessary. This not only amplifies cognitive burden, but also makes precise state construction substantially less likely. Even without explicit distractors, LLM performance can already degrade as context length grows [4, 17]; in branch-oriented test generation, this effect is further magnified because the relevant context is often deeply buried inside much longer method bodies. Taken together, these two issues suggest a systematic form of *cognitive misalignment* in LLM-based unit test generation: the information shown to the model is often misaligned with the information actually needed to cover a target branch. This misalignment has two complementary dimensions, namely *what to satisfy* and *what to attend to*.

To address this cognitive misalignment, we propose COGPATH, an LLM-driven framework for automated unit test generation that explicitly tackles both dimensions of the gap, by making critical constraints explicit and concentrating the model’s attention on branch-relevant program context. First, **Constraint-Hints** addresses the *what to satisfy* dimension by making the execution conditions of uncovered paths explicit. Instead of forcing the LLM to deduce implicit constraints from raw code, this mechanism extracts path conditions and translates them into actionable guidance. It drastically reduces the model’s reasoning difficulty by directly suggesting *how* to satisfy a target condition—such as invoking specific

setter methods to establish the required object state. By surfacing these hidden cross-method preconditions, it actively guides the LLM to construct valid initialization sequences, ensuring complex dependencies are properly addressed rather than silently skipped. Then, *coverage-driven backward slicing* addresses the *what to attend to* dimension. Rather than supplying the full method body, the slicer isolates the minimal set of statements that genuinely influence an uncovered branch, including cross-method dependencies such as field initializations and setter methods. This restructures the prompt context around branch-relevant dependencies, preventing the model's attention from being diffused across irrelevant code. The slicing is triggered selectively when coverage plateaus, targeting branches that resist standard generation. Both components are integrated into an iterative loop of generation, execution, and repair.

Evaluation. We evaluate COGPATH on 14 Java projects from Defects4J v2.0.1, covering 130 classes and 2971 methods with cyclomatic complexity greater than 10. Baselines include SymPrompt, HITS, and Panta, representing complementary design philosophies. COGPATH achieves 44.41% average branch coverage, outperforming SymPrompt (21.14%), HITS (34.24%), and Panta (32.32%). Gains are most pronounced on logic-heavy subjects with rich cross-method dependencies: on Gson, COGPATH reaches 41.24% versus SymPrompt's 3.59%; on Jsoup, it improves over HITS by 36.4%.

Ablation study confirms the complementary roles of both components. Removing *Backward Slicing* drops coverage from 44.41% to 38.47%, with the largest degradation on context-heavy subjects (e.g., Csv: 52.29%→24.80%). Removing *Constraint-Hints* drops coverage to 39.79%, most severely on subjects with object-state dependencies (e.g., JacksonDatabind: 17.85%→9.33%). Finally, COGPATH generalizes across backbone LLMs, achieving 44.41% (QC3-30B), 83.47% (QW3.5-397B), and 80.12% (DS-V3.1), confirming that its benefit derives from restructuring branch-relevant information rather than relying on any specific model.

In summary, this paper makes the following contributions:

- We identify a key limitation of existing LLM-based test generation approaches in handling implicit, cross-method constraints.
- We propose *Constraints-Hints*, a mechanism that makes path conditions explicit and guides LLM reasoning.
- We introduce a coverage-driven backward slicing technique to eliminate irrelevant context and expose critical dependencies.
- We design an iterative framework that integrates generation, execution feedback, and repair for sustained coverage improvement.
- We empirically demonstrate the effectiveness of our approach on real-world software systems.

2 Motivation

Recent LLM-based unit test generation approaches, such as SymPrompt [14], HITS [18], Panta [7], and JUnitGenie [9], have significantly reduced the manual effort required for testing complex methods. However, even with path-aware prompting and iterative refinement, a substantial gap in branch coverage remains.

We argue that this limitation stems from a fundamental *cognitive misalignment* between the information provided to the LLM and the information required to cover a target branch. Specifically,

effective branch coverage requires (i) satisfying implicit execution constraints and (ii) focusing on the minimal relevant context. Existing approaches, however, expose raw method bodies and implicitly expect the model to solve both problems simultaneously, leading to systematic failures.

We characterize this misalignment along two complementary dimensions.

Challenge 1: Underspecified Constraints (What to Satisfy).

Branch reachability often depends on implicit object states established through inter-procedural interactions [?]. These constraints are not explicitly represented in the input, forcing the LLM to infer them from incomplete information.

Consider the `addValue()` method from Apache Commons Math (Listing 1). The target branch depends on the condition `meanImpl != mean`, which can only be satisfied if a prior method modifies the internal state:

Listing 1: Target branch with hidden cross-method dependency

```
1 public void addValue(double value) {
2     ...
3     if (meanImpl != mean) { // target branch
4         meanImpl.increment(value);
5     }
6 }
```

When provided only with the method body, the LLM typically generates:

Listing 2: LLM-generated test (constraint not satisfied)

```
1 SummaryStatistics stats = new SummaryStatistics();
2 stats.addValue(1.0);
```

Although the test is executable, it fails to cover the branch because the required object-state constraint is never established. The correct test must explicitly satisfy the hidden precondition:

Listing 3: Correct test (constraint satisfied)

```
1 stats.setMeanImpl(new CustomMeanImpl());
2 stats.addValue(1.0);
```

This example illustrates a general failure mode: when constraints are encoded as cross-method state transitions, they remain invisible unless explicitly surfaced, making them unlikely to be satisfied during generation.

Challenge 2: Misaligned Context (What to Attend To). Even when all necessary conditions are locally available, they are often embedded within large methods containing substantial irrelevant logic. Providing the full method body introduces noise that is not aligned with the coverage objective.

Consider the `collect()` method from JacksonDatabind, where the target branch is guarded by a conjunction of three conditions:

Listing 4: Target branch with multi-variable condition

```
1 if (checkJavaLangObject && (_intr != null) && !
   methods.isEmpty()) { ... }
```

Each condition originates from a different part of the method body. The full method interleaves the initialization of methods, the resolution of `checkJavaLangObject`, and several loops and nested

branches entirely unrelated to the target guard. When exposed to the complete implementation, the LLM fails to identify that `_mixInResolver` is the critical dependency, and generates a test that leaves `checkJavaLangObject` permanently false:

Listing 5: LLM-generated test (branch not covered)

```
1 Collector collector = new Collector(intr, null);
  // _mixInResolver is null
2 collector.collect(typeFactory, tc, mainType,
  superTypes, primaryMixin);
3 // checkJavaLangObject remains false; target
  branch never reached
```

By contrast, applying coverage-driven backward slicing retains only the statements that bear data or control dependence on the target branch, and folds the rest into abstract pseudocode:

Listing 6: Branch-relevant slice

```
1 Map<MemberKey, MethodBuilder> methods = new
  LinkedHashMap<>();
2 _addMemberMethods(tc, mainType.getRawClass(),
  methods, primaryMixin);
3 for (...) { ... } // superTypes iteration -- no
  dependence on target branch
4 boolean checkJavaLangObject = false;
5 if (_mixInResolver != null) {
6   Class<?> mixin = _mixInResolver.
    findMixinClassFor(Object.class);
7   if (mixin != null) {
8     _addMethodMixIns(...);
9     checkJavaLangObject = true; // sole setter of
      guard variable
10  }
11 }
12 if (checkJavaLangObject && (_intr != null) && !
  methods.isEmpty()) { ... }
```

This focused representation makes the three guard dependencies structurally visible: `checkJavaLangObject` requires a non-null `mixin` for `Object.class`, `_intr` is a constructor-injected field, and `methods` is populated by `_addMemberMethods`. The aligned context enables the LLM to correctly configure the resolver and construct a valid initialization sequence:

Listing 7: LLM-generated test with aligned context

```
1 Collector collector = new Collector(intr, resolver
  );
2 // resolver returns non-null mixin ->
  checkJavaLangObject = true
3 // _addMemberMethods populates methods -> !methods
  .isEmpty()
4 collector.collect(typeFactory, tc, mainType,
  superTypes, primaryMixin);
5 // target branch covered
```

This reflects a second failure mode: even when constraints are present in the source, they are not structurally aligned with the model’s attention, making precise reasoning difficult.

3 Methodology

3.1 Overview

COGPATH is a coverage-guided iterative framework for automated unit test generation, illustrated in Figure 1. Given a method under test, COGPATH first performs an initialization step that extracts project-level dependencies, resolves the required JUnit imports, and seeds the suite with a syntactically valid scaffolding test containing a trivial `assertTrue(true)` assertion, ensuring a compilable starting point for all subsequent iterations. It then constructs the CFG, enumerates feasible paths, and collects dynamic coverage feedback to identify uncovered branches before entering an iterative refinement loop.

Each iteration begins with path selection, where *Constraint-Hints* analyzes the selected uncovered path and injects explicit branch conditions as structured guidance into the generation prompt. When coverage improvement stagnates, *Backward Slicing* is activated, replacing the full method body with the minimal set of statements that bear data or control dependence on the target branch. The LLM then generates candidate tests against this focused context; passing tests are merged into the suite while failing ones are forwarded to the repair phase. During repair, compiler diagnostics—including error messages and their associated line numbers—are incorporated into the repair prompt, allowing the LLM to precisely locate and correct the offending statements rather than re-generating the test from scratch. Coverage is re-measured at the end of each iteration to drive target selection in the next round. The loop terminates when the coverage target is reached, the iteration budget is exhausted, or coverage fails to improve for a configurable number of consecutive iterations.

3.2 Constraints-Hints

Constraints-Hints addresses the implicit precondition problem by decoupling constraint inference from test synthesis. Instead of requiring the LLM to simultaneously infer branch reachability and generate tests, we explicitly extract the conditions necessary to trigger a target branch and provide them as structured guidance.

While symbolic execution can derive precise path conditions, it often struggles to scale to real-world programs due to path explosion, complex heap interactions, and challenges in modeling external libraries and reflection [2?]. Rather than performing exact constraint solving, we aim to extract *sufficient* conditions that characterize branch reachability.

Concretely, given an uncovered branch, we analyze its predicate and derive a set of constraint hints that describe the required program state (e.g., variable relations, nullness conditions, and key object properties). These hints are then embedded into the generation prompt, guiding the LLM to construct inputs and invocation sequences that satisfy the target condition.

By transforming implicit dependencies into explicit guidance, *Constraints-Hints* reduces the reasoning burden on the LLM and enables more reliable generation of tests that reach previously uncovered branches.

Path formulation. Let $P = \langle b_1, b_2, \dots, b_k \rangle$ denote the sequence of branch decisions along a target execution path π , where each

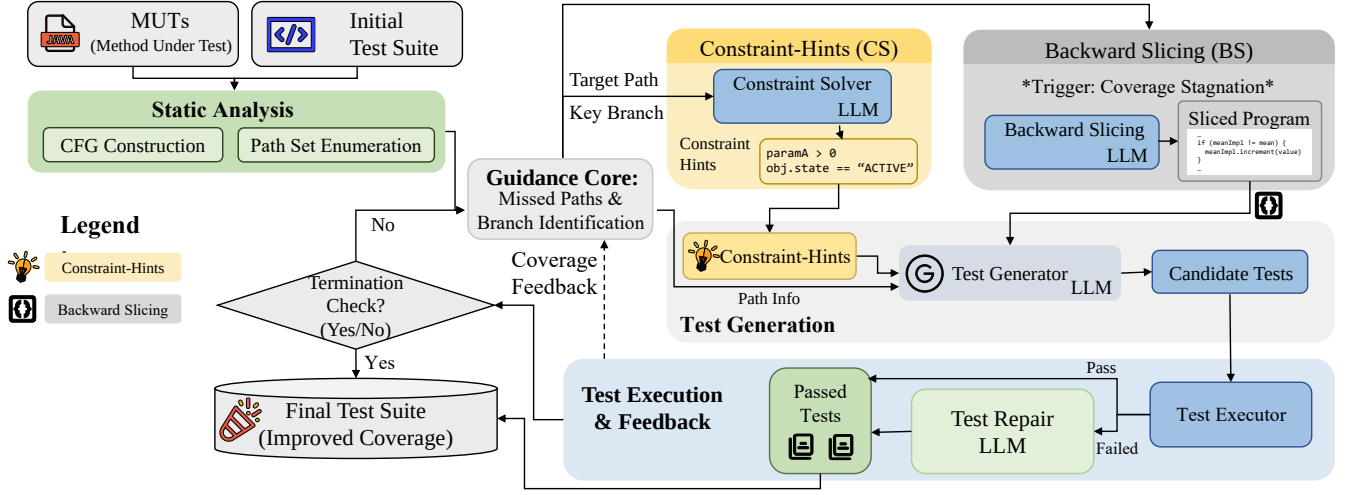


Figure 1: Overview of the coverage-guided test generation framework.

predicate $b_i(\mathbf{x}) \in \{\text{true}, \text{false}\}$ is evaluated over the program input vector $\mathbf{x}$. Covering the path requires an input assignment $\mathbf{x}^*$ that satisfies all branch predicates along the path:

$$\mathbf{x}^* \in \mathcal{F}(\pi) = \{\mathbf{x} \mid b_1(\mathbf{x}) \wedge b_2(\mathbf{x}) \wedge \dots \wedge b_k(\mathbf{x})\}. \quad (1)$$

Rather than solving this constraint system symbolically—which can be computationally expensive and brittle under complex object states or indirect method calls—COGPATH employs an LLM-based constraint solver to approximate the feasible input space as a light-weight set of interpretable conditions:

$$\mathcal{C}(\pi) = \{c_j \mid c_j \text{ is a necessary condition on } \mathbf{x}\}. \quad (2)$$

The first uncovered branch along the path, denoted as b^* , is identified from the coverage report and provided to the solver as additional context, guiding the model toward the most immediate obstacle preventing the path from being executed.

Integration. The constraint-solving prompt is generated from a configurable template that incorporates the source code, the branch sequence of π , and the uncovered branch b^* . The solver produces $\mathcal{C}(\pi)$ as a list of conditions such as `paramA > 0` or `obj.status == "ACTIVE"`. These conditions are appended to the test generation prompt, transforming an opaque path coverage objective into a concrete specification of required inputs.

Example. Consider the `addValue` method, which contains three conditional branches dispatching to overridable statistical implementations. An existing test suite covers only the false branches of all three predicates, meaning `meanImpl`, `varianceImpl`, and `geMeanImpl` all refer to their default implementations. The first uncovered branch is `(meanImpl != mean)`. The constraint solver identifies that satisfying this predicate requires replacing the default implementation before invoking `addValue`. This object-state precondition is expressed as a constraint in $\mathcal{C}(\pi)$:

$$c : \text{meanImpl} \neq \text{mean}$$

which implies that a call such as `setMeanImpl(...)` must occur prior to invoking `addValue`. By making this requirement explicit, the test generation LLM can construct the correct setup sequence and cover the previously unreachable branch.

3.3 Backward Slicing

Coverage improvement does not always progress smoothly: certain branches remain persistently uncovered even after multiple iterations, typically because their reachability depends on subtle state dependencies that standard generation fails to capture. When such a plateau is detected, COGPATH triggers a backward slicing step targeted at these stubborn branches, replacing the full method body with the minimal context actually relevant to the uncovered path.

Slice extraction. Given an uncovered path π , the framework extracts the subset of statements that may influence the execution of that path:

$$\text{Slice}(P, \pi) = \{s_i \in P \mid s_i \rightsquigarrow \pi\}. \quad (3)$$

A dedicated LLM is then invoked with the uncovered code segment, the focal method source code, and dependency context from related classes. The model returns a structured response containing the relevant slice statements, required input values, necessary object states, and potential setup operations.

Test generation from slices. The extracted slice information is supplied as additional context to the test generation prompt. The LLM then generates targeted unit tests that incorporate the identified prerequisites directly. These tests are compiled, executed, and measured using the same validation pipeline as regular test candidates, allowing the framework to recover from coverage plateaus while maintaining the overall iterative workflow.

Algorithm 1: Iterative Test Generation with Constraints-Hints and Backward Slicing

Input: *src*: source file, *suite*: initial test suite (possibly empty)
Output: Augmented test suite with improved coverage

```

deps ← EXTRACTDEPS();
cfg, methodDict ← BUILDCFG(src);
noIncr ← 0, iter ← 0;
cov, missed ← RUNCOVERAGE(suite);
while iter < maxIter and noIncr <  $\Lambda$  and cov <  $\tau$  do
    // Path selection and prompt construction
    if cov = 0 or iter = 0 then
        prompt ← BUILDBASELINEPROMPT(src, suite, deps);
        ; // baseline prompt
    else
         $\Pi$  ← SELECTPATHS(cfg, methodDict, missed);
         $\mathcal{C}$  ←  $\emptyset$ ;
        foreach path  $\pi \in \Pi$  do
             $b^*$  ← FIRSTUNCOVEREDBRANCH( $\pi$ , missed);
             $\mathcal{C}(\pi)$  ← CONSTRAINTSOLVER(src,  $\pi$ ,  $b^*$ );
            ; // Constraints-Hints
        end
        prompt ← BUILDPROMPT(src, suite, deps,  $\Pi$ ,  $\mathcal{C}$ );
    end
    // Test generation
     $\mathcal{T}$  ← GENERATETESTS(prompt);
    // Backward slicing for stubborn uncovered paths
    if noIncr  $\geq \beta \cdot \Lambda$  then
        slices ← BACKWARDSLICER(src, missed);
         $\mathcal{T}$  ←  $\mathcal{T} \cup \text{GENERATEFROMSLICES}(\text{slices}, \text{suite}, \text{deps})$ ;
    end
    // Validation and repair
    failed ← {};
    foreach t  $\in \mathcal{T}$  do
        if VALIDATETEST(t, suite) then suite ← suite  $\cup \{t\}$ ;
        else failed ← failed  $\cup \{t\}$ ;
    end
    if failed  $\neq \emptyset$  then
        suite ← suite  $\cup \text{REPAIRANDVALIDATE}(\text{failed}, \text{src}, \text{suite}, \text{deps})$ ;
    end
    // Coverage update and termination check
    (cov', missed') ← RUNCOVERAGE(suite);
    if cov' > cov then noIncr ← 0;
    else noIncr ← noIncr + 1;
    cov ← cov', missed ← missed', iter ← iter + 1;
end
return suite;

```

3.4 Iterative Workflow of CogPATH

The complete workflow of CogPATH is formalized in Algorithm 1. The framework accepts the source file and an initial test suite as input, and iteratively generates, validates, and repairs tests until a coverage target is met or a termination condition is triggered.

Initialization. The framework extracts test-scope dependencies, constructs the CFG and method dictionary from the source file, and measures baseline coverage on the initial suite (lines 1–4).

Path selection with Constraints-Hints. At each non-initial iteration, SELECTUNCOVEREDPATHS identifies candidate paths from the CFG that intersect with missed lines or branches, weighted by both coverage impact and visit history *pathHistory* (line 8). For each selected path π , the constraint solver derives $\mathcal{C}(\pi)$ by analyzing the branch sequence up to the first uncovered predicate b^* , and these conditions are embedded into the generation prompt (lines 9–13).

Backward slicing. When *noIncr* $\geq \beta \cdot \Lambda$, indicating a coverage plateau, the backward slicer is invoked on the remaining missed code segments. It returns structured prerequisite information—required inputs, object states, and method mocks—which is used to construct an additional batch of targeted tests $\mathcal{T}_s$ (lines 16–19).

Validation and repair. Each generated test is individually inserted into the suite and executed. Failing tests are collected with their error messages and forwarded to the repair phase, which re-invokes the LLM with failure feedback to produce corrected versions. Successfully validated repaired tests are also incorporated into the suite (lines 21–32).

Termination. The loop exits when the coverage target τ is reached, the maximum iteration count *maxIter* is exhausted, or coverage fails to improve for Λ consecutive iterations.

4 EXPERIMENTS**4.1 Experimental Setup**

In this section, we first introduce the construction and statistics of the evaluation dataset.

Dataset. To evaluate the effectiveness of CogPATH in generating unit tests for complex methods, we utilize real-world subjects from the Defects4J (v2.0.1) benchmark [8]. We selected the latest fixed version of each subject program to ensure compatibility with modern build environments. Following prior work [7], we excluded projects that do not use Maven or rely on deprecated dependencies (i.e., Chart, Mockito, and Closure). The remaining 14 subjects are compatible with Java 8+, Maven (v3.6.3), and JUnit (v4.13.2). To specifically assess performance on logic-heavy code, we focus on complex classes rather than entire projects. We identified non-abstract, public classes containing at least one method with Cyclomatic Complexity (CC) > 10 [10]. The statistics of our evaluation dataset are summarized in Table 1.

Evaluation Metrics. To evaluate the effectiveness of the generated unit tests, we primarily focus on code coverage, which is the standard metric in automated test generation research [5, 12]. We report two specific metrics:

- **Line Coverage (LC):** The ratio of executable lines in the target class that are exercised by the generated test suite. It provides a baseline measure of how much of the implementation has been reached.
- **Branch Coverage (BC):** The ratio of executed decision outcomes (e.g., the true and false paths of an *if* statement) to the total number of possible branches. Given our focus on methods with high Cyclomatic Complexity (CC > 10), branch coverage is our primary metric, as it more accurately reflects the tool's ability to navigate complex logical structures and boundary conditions.

Table 1: Dataset Metrics with #MUTs

Identifier	Project	Version	#C	#MUTs	CC_{max}	CC_{min}	CC_{med}
Cli	Cli-40f	1.5-SNAP	2	31	11	11	11
Codec	Codec-18f	1.11-SNAP	7	78	31	11	13
Collections	Collections-28f	4.2-SNAP	5	218	32	11	16
Compress	Compress-47f	1.16-SNAP	9	65	22	11	13
Csv	Csv-16f	1.6-SNAP	3	72	25	11	14
Gson	Gson-16f	2.8.2-SNAP	4	75	37	11	22
JCore	JacksonCore-26f	2.10.0-SNAP	9	161	30	11	14
JDatabind	JacksonDatabind-112f	2.9.9	9	371	22	11	15
JXml	JacksonXml-5f	2.9.6-SNAP	4	118	29	12	22
Jsoup	Jsoup-93f	1.12.2-SNAP	8	171	22	11	12
JxPath	JxPath-22f	1.4-SNAP	12	115	32	11	13
Lang	Lang-4f	3.2-SNAP	17	586	30	11	16
Math	Math-2f	3.3-SNAP	30	452	31	11	14
Time	Time-13f	2.1	11	458	30	11	17
Total/Avg			130	2971	27.4	11.1	15.1

All coverage data in our experiments are collected using the JaCoCo library, ensuring consistent and reproducible measurements across all subjects and baselines.

Research Questions. To evaluate the effectiveness and design of CogPATH, we address the following three research questions:

- **RQ1: Effectiveness of CogPATH.** How does CogPATH perform compared to baselines in terms of line and branch coverage, especially on complex, logic-heavy methods?
- **RQ2: Ablation Study.** What are the individual contributions of Backward Slicing and Constraint Hints to the overall coverage of CogPATH, and how does the iterative generation loop affect the branch coverage?
- **RQ3: Model Generalizability of CogPATH.** Is CogPATH's effectiveness robust across different underlying large language models, and how do their performances vary?

Baselines. To evaluate the performance of CogPATH, we compare it against three state-of-the-art LLM-based unit test generation frameworks representing diverse paradigms in the field:

- **SymPrompt [14]:** A prompt-engineering approach that encodes symbolic execution concepts into structured prompts. It extracts branch conditions from Control Flow Graphs (CFGs) and constructs path-oriented prompts to guide the LLM in generating inputs that satisfy specific execution paths.
- **HITS [18]:** A hierarchical prompting framework that decomposes complex methods into smaller code segments before test generation. It adopts a self-healing strategy that iteratively repairs failing tests to improve compilability and coverage.
- **Panta [7]:** A hybrid framework that combines static CFG analysis with dynamic coverage feedback to identify uncovered paths. It constructs context-aware prompts that expose intra-class data dependencies to guide the LLM toward uncovered branches.

Due to missing components in the official HITS repository, we re-implemented its workflow by strictly following the prompt templates described in the paper and the procedural logic of available open-source fragments. For fair comparison, all methods use the same backbone model as CogPATH in each experiment. In RQ1 and RQ2, we adopt Qwen3-Coder-30B-A3B (QC3-30B), deployed locally on an NVIDIA H800 GPU (80 GB) for cost-efficient evaluation, while RQ3 further includes Qwen3.5-397B-A17B, GPT-5.4-Mini, and DeepSeek-V3.1 to assess generalizability. For each MUT, all approaches are restricted to the declaring class source code and import statements, without access to project-level dependencies or configurations, ensuring that performance differences stem from the frameworks rather than external information. To improve reproducibility, all models are configured with temperature 0.2 and top- p 1.0. The key hyperparameters of CogPATH are set as follows: $maxIter = 20$, $\Lambda = 3$, $\beta = 0.6$, and target coverage $\tau = 100.00\%$, with all experiments starting from an empty test suite.

4.2 RQ1: Effectiveness of Our Approach

To answer RQ1, we compare CogPATH against three state-of-the-art baselines on the selected dataset. Results are summarized in Table 2.

4.2.1 Overall Performance. As shown in Table 2, CogPATH achieves the highest average LC and BC across all projects. Specifically, it attains an average BC of 44.41%, outperforming SymPrompt (21.14%), HITS (34.24%), and Panta (32.32%). Same trends hold for LC, where CogPATH achieves 53.14% versus SymPrompt (26.19%), HITS (40.31%), and Panta (39.77%).

Gains over the two strongest baselines—HITS (+10.2%) and Panta (+12.1%)—confirm that jointly reducing irrelevant context and making implicit preconditions explicit addresses coverage gaps that iterative refinement and context-aware prompting alone cannot close. The margin widens substantially against SymPrompt (+23.3%), underscoring the necessity of explicit program analysis when navigating logic-heavy control flows.

Table 2: Coverage Across Approaches

Project	#C	CC_{avg}	SymPrompt		HITS		Panta		CogPATH	
			LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	2	11.00	17.93	9.02	73.50	59.58	63.11	37.98	84.32	70.73
Codec	7	16.14	29.68	24.96	65.73	60.25	60.53	54.82	76.97	69.60
Collections	5	18.60	32.63	32.42	68.23	70.00	33.44	32.40	40.46	34.63
Compress	9	15.33	11.84	11.34	31.64	25.24	34.36	28.45	36.09	29.05
Csv	3	16.67	37.69	23.79	28.95	14.73	50.98	37.99	62.27	52.29
Gson	4	23.25	13.16	3.59	31.54	21.25	21.81	15.20	63.82	41.24
JCore	9	16.33	23.62	21.44	19.85	18.51	26.24	21.97	41.72	35.54
JDataBind	9	15.33	9.99	6.06	18.33	14.44	18.17	14.34	23.35	17.85
JXml	4	21.50	17.45	18.56	18.59	18.84	31.89	23.55	20.61	17.08
Jsoup	8	14.13	24.72	19.13	27.82	17.36	49.80	39.25	66.43	52.55
JXPath	12	15.50	12.01	9.48	23.11	22.76	34.10	29.35	42.28	35.84
Lang	17	16.88	43.98	40.42	62.82	59.77	39.26	36.61	59.21	52.14
Math	30	16.47	38.55	33.70	30.87	26.38	46.40	39.16	59.60	54.02
Time	11	17.36	53.37	42.06	63.32	50.23	46.74	41.42	66.79	59.25
Average			26.19	21.14	40.31	34.24	39.77	32.32	53.14	44.41

4.2.2 *Performance on High-Complexity Projects.* The advantages of CogPATH are most pronounced on projects with high cyclomatic complexity and deeper inter-method dependencies.

For instance, on Gson ($CC_{avg} = 23.25$, the highest in the benchmark), CogPATH achieves a BC of 41.24% while SymPrompt reaches only 3.59%; on JCore ($CC_{avg} = 16.33$), BC rises from 18.51% (HITS) to 35.54%; and on Jsoup ($CC_{avg} = 14.13$), BC improves from 17.36% (HITS) to 52.55%. These gains collectively demonstrate that reducing irrelevant context via backward slicing and surfacing implicit preconditions via constraint hints are both essential for navigating the dense branching logic and cross-method dependencies that characterize high-complexity projects.

These results indicate that CogPATH excels particularly in logic-heavy methods where complex object interactions and conditional dependencies are prevalent.

Answer to RQ1

CogPATH outperforms all three baselines on average, achieving a BC of 44.41%, representing improvements of 10.2%–23.3% over baselines. Its advantage is most pronounced in projects with high cyclomatic complexity and cross-method dependencies (e.g., +20% on Gson, +14% on JCore).

4.3 RQ2: Ablation Study

To examine the design of CogPATH, this section addresses two questions: (1) what do BS and CS each contribute to final coverage, and (2) does the iterative generation loop meaningfully accelerate coverage accumulation beyond the gains from individual components?

4.3.1 *Contribution of BS and CS.* We construct three variants—**w/o BS** (removing backward slicing), **w/o CS** (removing constraint

hints), and **w/o CS+BS** (removing both)—all sharing the same iterative loop limit (maxIter=20) and evaluated under QC3-30B. Results are assessed via Table 3.

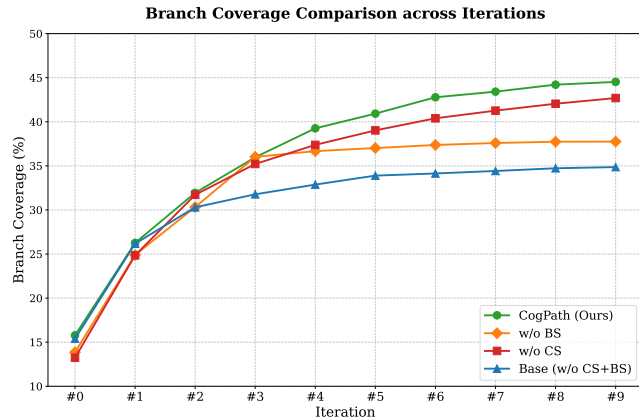
Table 3 shows that both components contribute independently: removing BS alone drops average BC by 5.94 % (44.41% → 38.47%), removing CS alone by 4.62 % (44.41% → 39.79%), and removing both by 7.84 pp (44.41% → 36.57%). The joint degradation falls below the sum of individual losses (10.56 pp), a subadditivity that reflects partial functional overlap—CS not only makes cross-method preconditions explicit but also implicitly focuses the LLM on branch-relevant predicates, partially mirroring BS’s context-reduction role—though neither component can fully substitute for the other. The same degradation ordering holds for LC (BS removal: −6.63 pp; CS removal: −6.56 pp; both: −9.70 pp).

Yet these averages mask substantial per-project variation: BS and CS address structurally distinct bottlenecks, making them complementary rather than redundant. BS contributes most where method bodies are dominated by branch-irrelevant statements: Csv is a highly configurable, multi-dialect parser whose methods interleave handling for delimiters, quotes, escape sequences, and EOF within a single control flow. Backward slicing strips these dialect-processing branches that are irrelevant to the target predicate, letting the LLM focus on the right conditions; without it, the interference from unrelated branches collapses BC from 52.29% to 24.80% (−27.49 %). CS contributes most where branch reachability is governed by cross-call preconditions rather than local logic: in *JDataBind*, reaching many branches requires a specific sequence of configuration API calls to establish the correct mapper or reader/writer state—constraints that are invisible within the focal method body alone. CS makes these preconditions explicit; without it, the LLM readily produces syntactically valid tests that nonetheless fail to satisfy the required object state, reducing BC from 17.85% to 9.33% (−8.52 %). In short, BS addresses context noise while CS addresses hidden state dependencies—distinct bottlenecks that neither component can resolve for the other.

Table 3: Ablation Study of CogPATH Components

Project	CC_{avg}	CogPath		w/o CS		w/o BS		w/o CS+BS	
		LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	11.00	84.32	70.73	84.74	68.75	68.69	57.68	72.62	56.54
Codec	16.14	76.97	69.60	54.86	50.68	66.77	57.94	66.34	56.53
Collections	18.60	40.46	34.63	39.51	38.59	32.42	36.60	37.43	34.45
Compress	15.33	36.09	29.05	31.22	28.58	25.43	19.04	23.45	18.11
Csv	16.67	62.27	52.29	46.18	36.38	36.47	24.80	60.54	47.07
Gson	23.25	63.82	41.24	43.94	27.73	49.40	31.85	40.92	26.52
JCore	16.33	41.72	35.54	30.58	27.31	45.39	39.20	22.97	21.07
JDatabind	15.33	23.35	17.85	12.44	9.33	33.74	26.55	19.59	15.54
JXml	21.50	20.61	17.08	23.80	18.61	35.30	26.84	26.79	23.08
Jsoup	14.13	69.11	53.76	61.48	46.64	69.11	53.76	59.00	47.11
JxPath	15.50	42.28	35.84	47.56	41.96	28.06	22.35	34.94	31.44
Lang	16.88	59.21	52.14	66.05	63.04	56.84	51.38	57.12	53.91
Math	16.47	59.60	54.02	63.42	56.61	38.60	32.66	43.23	39.90
Time	17.36	66.79	59.25	46.29	42.81	64.99	57.89	43.21	40.65
Average		53.14	44.41	46.58	39.79	46.51	38.47	43.44	36.57

This benefit, however, is not universal: on *JCore* and *JXml*, removing BS raises coverage, pointing to two related failure modes. First, many branch predicates in these projects are feature-flag checks whose controlling field is set via a builder chain at construction time, not passed as a method argument. BS correctly traces the data flow to the field, but the full assignment path spans multiple factory-method layers; the resulting slice is semantically incomplete rather than merely short, and the LLM is left with worse context than the unsliced method body. Second, *JXml* compounds this with a two-layer configuration stack—a Woodstox *WstxInputFactory* beneath Jackson’s *XmlMapper*—so BS structurally misses the lower layer’s dependencies from the start, not merely truncates them.

**Figure 2: Branch coverage improvement over iterations for CogPATH and its three ablated variants.**

4.3.2 Improvement of Iterative Framework. Beyond final coverage, we examine whether the iterative loop itself accelerates progress.

We ran all four configurations (CogPATH and its three ablated variants) under QC3-30B, recording average BC at each step; the resulting growth curves are shown in Figure 2. For clarity, we plot the first 10 iterations only, because all curves become nearly flat afterward and additional rounds yield negligible gains.

Across the entire range, CogPATH remains the top-performing configuration at every iteration, showing both the fastest early-stage growth and the highest eventual plateau. In contrast, w/o BS and w/o CS rise more slowly and saturate at lower coverage levels, while w/o CS+BS plateaus the earliest with the lowest ceiling. This three-way pattern—faster rise, higher ceiling, and later plateau—indicates that BS and CS improve iterative testing in two complementary ways: they increase the effectiveness of each generation round and they postpone premature saturation.

Answer to RQ2

Both BS and CS make independent, complementary contributions and neither component is dispensable: BS eliminates context noise (−5.94 % on average), while CS makes implicit cross-method preconditions explicit (−4.62 % on average). The iterative loop further amplifies these gains: CogPATH leads at every iteration with the fastest early growth and the latest saturation.

4.4 RQ3: Model Generalizability of CogPATH

To evaluate model generalizability, we run CogPATH under four backbone models—QC3-30B (used in RQ1 and RQ2), GPT-5.4-Mini, QW3.5-397B, and DS-V3.1—using the same experimental configuration as RQ1. Results are summarized in Table 4.

Overall trends. Scaling up the backbone model brings substantial coverage improvements across all 14 projects. While QC3-30B achieves an average BC of 44.41%, the three large-scale models reach 79.76% (GPT-5.4-Mini), 80.36% (DS-V3.1), and 83.47% (QW3.5-397B)—gains of 35–39 % over QC3-30B. Among them, QW3.5-397B

Table 4: Model Generalization Across Different LLMs

Project	#C	CC	QC3-30B		GPT-5.4-Mini		QW3.5-397B		DS-V3.1	
			LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)	LC(%)	BC(%)
Cli	2	11.00	84.32	70.73	97.86	86.18	98.82	92.63	98.82	91.93
Codec	7	16.14	76.97	69.60	89.78	82.61	98.32	95.30	93.60	88.46
Collections	5	18.60	40.46	34.63	98.31	94.14	98.16	94.43	98.02	94.20
Compress	9	15.33	36.09	29.05	87.15	80.29	86.92	80.90	81.32	76.04
Csv	3	16.67	62.27	52.29	87.42	77.76	95.71	87.42	85.70	77.53
Gson	4	23.25	63.82	41.24	90.26	83.06	92.88	87.97	92.13	82.86
JCore	9	16.33	41.72	35.54	63.71	57.83	83.25	76.77	66.62	57.76
JDatabind	9	15.33	23.35	17.85	76.16	68.78	74.93	69.81	84.06	77.44
JXml	4	21.50	20.61	17.08	73.41	74.20	62.20	63.65	66.06	65.22
Jsoup	8	14.13	66.43	52.55	93.37	82.96	95.87	87.49	87.15	79.98
JXPath	12	15.50	42.28	35.84	81.72	75.11	71.30	66.22	76.89	73.21
Lang	17	16.88	59.21	52.14	90.65	84.32	91.72	87.44	88.21	85.02
Math	30	16.47	59.60	54.02	88.89	81.82	92.96	87.47	91.69	85.41
Time	11	17.36	66.79	59.25	94.18	87.63	96.94	91.08	94.71	89.92
Average			53.14	44.41	86.63	79.76	88.57	83.47	86.07	80.36

achieves the highest overall BC, consistent with its leading performance on code-oriented benchmarks documented in its technical report [?]. Notably, even QC3-30B already surpasses every RQ1 baseline (best: HITS at 34.24%), confirming that CogPath’s framework-level contributions hold independently of model scale.

Where model scale matters most. The capacity-driven gains are pronounced on projects that demand dense conditional reasoning. On JDatabind, QC3-30B reaches only 17.85% BC while large models achieve 68–77%. Cross-referencing with the RQ2 ablation sharpens this picture: removing CS costs 8.52% on JDatabind, while removing BS *improves* coverage by 8.70 %. Together, these results indicate that JDatabind’s dense cross-method initialization logic depends critically on explicit constraint guidance rather than code-context breadth. It is precisely this constraint reasoning and following capability—translating structured hints into correct multi-step object state construction—that scales with model size, explaining the pronounced gap between QC3-30B and the larger models on this project. By contrast, on structurally simpler projects such as Cli and Time, the gap narrows to under 20 %, where CogPath’s structural guidance already suffices for QC3-30B to handle the less demanding conditional logic.

Structural bottlenecks. Despite the strong overall gains, JCore and JXml remain below the large-model average of ~81% regardless of backbone—best BC of 76.77% and 74.20%, respectively. As identified in RQ2, these projects share a framework-level bottleneck: their branch predicates are governed by builder-chain configuration state that spans multiple factory-method layers, causing BS to produce semantically incomplete slices that are worse than the unsliced method body. Scaling up model capacity cannot compensate for structurally deficient input, which is why the ceiling persists even under the strongest models. The consequence is also visible in higher inter-model variance on these two projects (e.g., JCore: QW3.5-397B=76.77% vs. GPT-5.4-Mini=57.83%), suggesting that when CogPath’s structural guidance is compromised, performance may fall back on model-specific code reasoning rather than

shared framework scaffolding. Extending backward slicing to handle cross-layer builder chains is therefore the primary direction for future refinement.

Answer to RQ3

CogPath performs robustly across all four backbone LLMs, with stronger models consistently yielding higher coverage (79.76–83.47% avg. BC for large models). On high-complexity projects with deep inter-method dependencies, improved model reasoning ability further amplifies this advantage.

5 DISCUSSION

Inference Token Cost. To present cost in a deployment-relevant form, we report per-MUT cost under the four backbones used in RQ3: QC3-30B (0.003\$/MUT), GPT-5.4-Mini (0.019\$/MUT), QW3.5-397B (0.03\$/MUT), and DS-V3.1 (0.008\$/MUT). Across all four settings, the per-MUT cost remains low in absolute terms, indicating that the framework-level overhead of CogPath is operationally affordable.

Runtime Overhead. Across all 2,971 MUTs using QC3-30B as the backbone, CogPath averages 30.3 s/MUT, incurring +47% overhead over SymPrompt (20.6 s/MUT) and +13% over HITS (26.7 s/MUT), while running 4% faster than Panta (31.5 s/MUT). Against these time differentials, CogPath achieves a BC of 44.41% versus 21.14%, 34.24%, and 32.32% for the three baselines—relative improvements of +110%, +30%, and +37% respectively. The overhead is thus proportionate to the coverage gains in every comparison. Of the 30.3 s, ≈1.2 s is attributable to static preprocessing, which scales with the inter-procedural dependency graph and is already included in the figures above; the remainder is LLM inference time.

Limitations.

Threats to Validity. The effectiveness of COGPATH depends on the capabilities of the underlying LLM, and results may vary as more advanced models are introduced; we mitigate this by evaluating across four models spanning a range of scales and architectures (RQ3). Data contamination is a known concern in LLM-based software engineering research, as evaluated models may have encountered Defects4J code during pretraining; to reduce its effect, all methods are evaluated on the same projects with the same backbone model, so reported results reflect relative improvements between approaches rather than absolute figures. Our evaluation is further limited to Java projects from the Defects4J benchmark, which may affect generalizability to other languages or ecosystems, though the underlying techniques are language-agnostic in principle. Finally, our dataset is restricted to methods with $CC > 10$, as COGPATH specifically targets logic-heavy code; performance on lower-complexity methods has not been assessed, though COGPATH is intended to complement rather than replace existing approaches that already handle such cases well.

6 Related Work

6.1 Automated Unit Test Generation

Automated unit test generation has been studied through two major paradigms: search-based software testing (SBST) and, more recently, LLM-based synthesis. Early techniques such as Randoop [12] and EvoSuite [5] drive test generation by optimizing structural coverage objectives. EvoSuite applies evolutionary algorithms over whole test suites to maximize branch coverage and automatically infers regression assertions. While effective at exploring input spaces, SBST methods fundamentally operate on syntactic signals and struggle with semantic preconditions that require understanding object states or inter-procedural call sequences [11].

Before the rise of large language models, neural sequence-to-sequence models offered a first bridge between code understanding and test synthesis. ATLAS [?] frames test generation as a translation task, training an encoder-decoder network to predict assert statements from focal methods. AthenaTest [?] extends this direction by fine-tuning pre-trained models on large method-test corpora to automate full JUnit test synthesis. These deep learning approaches demonstrate that neural models can capture surface-level code patterns, but their capacity is fundamentally bounded by training distribution: they cannot reliably generalize to the combinatorial space of runtime behaviors or reason about execution semantics.

The advent of large language models overcomes this boundary by enabling open-ended test synthesis grounded in broad code understanding. ChatUniTest [3] introduces a focal-context mechanism that selectively exposes program information to the model and adopts a generate-validate-repair loop to improve compilation rates. These early LLM-based approaches demonstrate the potential of neural generation but expose a new challenge: without structured guidance, models tend to produce superficially plausible tests that fail to reach complex branches.

A first wave of follow-up work addresses this gap by incorporating execution signals into the generation loop. CoverUp [13] iteratively targets untested code regions using dynamic coverage reports, directing the model's attention toward uncovered lines, and

YATE [?] further repairs failing tests to improve validity. These methods raise coverage by closing the generation-execution loop, but still rely on what the model can observe locally in the method body, leaving cross-method dependencies implicit.

A second wave enriches the model's input with structural program information. SymPrompt [14] extracts branch conditions from Control Flow Graphs (CFGs) and embeds them as structured prompts, helping the model reason about execution paths. HITS [18] decomposes complex methods into smaller slices to reduce cognitive load, improving syntactic validity. Panta [7] combines static CFG analysis with dynamic coverage feedback to identify and target uncovered paths. JUnitGenie [9] further achieves strict path sensitivity by constructing minimal, path-specific contexts using type and data-dependency information. These advances substantially reduce the structural mismatch between model input and coverage objective, yet a fundamental gap persists on branches whose reachability depends on cross-method object states—preconditions established by prior setter or initializer calls that are invisible within the focal method body. Existing approaches conflate constraint inference with test synthesis in a single generation step, leaving the model to silently skip complex dependency chains. Furthermore, even when path information is provided, the full method body dilutes the model's attention with statements irrelevant to the target branch, amplifying context noise as complexity grows [4, 17]. COGPATH directly addresses both dimensions: Constraint-Hints makes implicit cross-method preconditions explicit before generation begins, while coverage-driven Backward Slicing restructures the prompt around branch-relevant dependencies.

6.2 Program Slicing

Program slicing [16, 19] extracts the subset of statements that influence a particular variable or program point. Classical static slicers identify data and control dependencies via program dependence graphs; JavaSlicer [6] extends this to Java programs by constructing precise inter-procedural dependence graphs over bytecode. These techniques have been widely applied in debugging, program comprehension, and regression testing. However, classical static slicers fundamentally require complete, parsable code: tools such as Joern fail on incomplete snippets due to missing type declarations, unresolved imports, and references to external libraries. NSSlicer [20] addresses this limitation by leveraging a pre-trained language model to learn fine-grained variable-statement dependency patterns, enabling static slicing of both complete and partial Java programs where traditional analysis is brittle. More recently, Shahandashti et al. [?] demonstrate that LLMs hold promise as program slicers for Java: evaluating GPT-4o, GPT-3.5 Turbo, Llama-2, and Gemma-7B with advanced prompting strategies, they find GPT-4o achieves the best static slicing accuracy of 60.84%, outperforming all other evaluated models. Nevertheless, complex control flow remains the dominant failure cause, indicating that precise dependency reasoning is still a fundamental challenge even for state-of-the-art LLMs.

In the test generation domain, HITS [18] applies a form of method decomposition to reduce LLM input complexity. However, its decomposition is driven by structural heuristics rather than data and

control dependencies, and is not targeted toward any specific uncovered branch; for high-complexity methods the number of generated segments can grow substantially, incurring token overhead regardless of coverage impact. In contrast, COGPATH adopts a stricter, coverage-driven interpretation of backward slicing: starting from an uncovered branch condition, the slicer traces all data and control dependencies—including cross-method field initializations and setter definitions—to produce a focused, branch-specific context. Slicing is triggered selectively only when coverage improvement stalls, ensuring that each slice is both necessary and precisely targeted, rather than a structurally-motivated but coverage-agnostic decomposition.

7 Conclusion and Future work

We presented COGPATH, an LLM-driven framework for automated unit test generation that addresses two complementary dimensions of cognitive misalignment: Constraint-Hints makes implicit cross-method preconditions explicit, while coverage-driven Backward Slicing eliminates irrelevant context to concentrate the model's attention on branch-relevant dependencies. Evaluated on 130 classes and 2,971 complex methods from 14 real-world Java projects, COGPATH achieves 44.41% average branch coverage, outperforming state-of-the-art baselines by up to 89.3% and generalizing consistently across LLMs of varying capacity. Future work will focus on adaptive slice granularity selection and extending constraint guidance to more complex object-oriented initialization patterns.

References

- [1] Roberto Baldoni, Emilio Coppa, Daniele Cono D'Elia, Camil Demetrescu, and Irene Finocchi. 2018. A Survey of Symbolic Execution Techniques. arXiv:1610.00502 [cs.SE] <https://arxiv.org/abs/1610.00502>
- [2] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. 2008. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, Vol. 8. 209–224.
- [3] Yinghao Chen, Zehao Hu, Chen Zhi, Junxiao Han, Shuiguang Deng, and Jianwei Yin. 2024. *ChatUnitTest: A Framework for LLM-Based Test Generation*. arXiv:2305.04764 <http://arxiv.org/abs/2305.04764>
- [4] Yufeng Du, Minyang Tian, Srikanth Ronanki, Subendhu Rongali, Sravan Bodapati, Aram Galstyan, Azton Wells, Roy Schwartz, Eliu A Huerta, and Hao Peng. 2025. Context Length Alone Hurts LLM Performance Despite Perfect Retrieval. arXiv:2510.05381 [cs.CL] <https://arxiv.org/abs/2510.05381>
- [5] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (Szeged, Hungary) (ESEC/FSE '11)*. Association for Computing Machinery, New York, NY, USA, 416–419. doi:10.1145/2025113.2025179
- [6] Carlos Galindo, Sergio Perez, and Josep Silva. 2022. A Program Slicer for Java (Tool Paper). In *Software Engineering and Formal Methods*, Bernd-Holger Schlingloff and Ming Chai (Eds.). Springer International Publishing, Cham, 146–151.
- [7] Sijia Gu, Noor Nashid, and Ali Mesbah. 2025. LLM Test Generation via Iterative Hybrid Program Analysis. doi:10.48550/arXiv.2503.13580 arXiv:2503.13580
- [8] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: a database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (San Jose, CA, USA) (ISSTA 2014)*. Association for Computing Machinery, New York, NY, USA, 437–440. doi:10.1145/2610384.2628055
- [9] Dianshu Liao, Xin Yin, Shidong Pan, Chao Ni, Zhenchang Xing, and Xiaoyu Sun. 2025. Navigating the Labyrinth: Path-Sensitive Unit Test Generation with Large Language Models. doi:10.48550/arXiv.2509.23812 arXiv:2509.23812
- [10] T.J. McCabe. 1976. A Complexity Measure. *IEEE Transactions on Software Engineering* SE-2, 4 (1976), 308–320. doi:10.1109/TSE.1976.233837
- [11] Phil McMinn. 2011. Search-Based Software Testing: Past, Present and Future. In *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. 153–163. doi:10.1109/ICSTW.2011.100
- [12] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Companion to the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications Companion* (Montreal, Quebec, Canada) (*OOPSLA '07*). Association for Computing Machinery, New York, NY, USA, 815–816. doi:10.1145/1297846.1297902
- [13] Juan Altmayer Pizzorno and Emery D. Berger. 2024. *CoverUp: Coverage-Guided LLM-Based Test Generation*. arXiv:2403.16218 <http://arxiv.org/abs/2403.16218>
- [14] Gabriel Ryan, Siddhartha Jain, Mingyue Shang, Shiqi Wang, Xiaofei Ma, Murali Krishna Ramanathan, and Baishakhi Ray. 2024. Code-Aware Prompting: A Study of Coverage-Guided Test Generation in Regression Setting Using LLM. *Proceedings of the ACM on Software Engineering* 1, FSE (July 2024), 951–971. doi:10.1145/3643769
- [15] The Apache Software Foundation. 2016. Apache Commons Math 3.6.1. <https://commons.apache.org/proper/commons-math/>. Version 3.6.1.
- [16] Frank Tip. 1995. A survey of program slicing techniques. *J. Program. Lang.* 3, 3 (1995). <http://compscinet.dcs.kcl.ac.uk/JIP/jp030301.abs.html>
- [17] Weiwei Wang, Jiyong Min, and Weijie Zou. 2026. Intelligence Degradation in Long-Context LLMs: Critical Threshold Determination via Natural Length Distribution Analysis. arXiv:2601.15300 [cs.CL] <https://arxiv.org/abs/2601.15300>
- [18] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (Sacramento CA USA, 2024-10-27)*. ACM, 1258–1268. doi:10.1145/3691620.3695501
- [19] Mark Weiser. 1984. Program Slicing. *IEEE Trans. Softw. Eng.* 10, 4 (July 1984), 352–357. doi:10.1109/TSE.1984.5010248
- [20] Aashish Yadavally, Yi Li, Shaohua Wang, and Tien N. Nguyen. 2024. A Learning-Based Approach to Static Program Slicing. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (April 2024), 83–109. doi:10.1145/3649814
- [21] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *arXiv preprint arXiv:2404.04966* (2024).